Assignment 2 (Solution) : Object Oriented Programming (Python)

1. **What is polymorphism in Python?**
   Polymorphism allows objects of different classes to be treated as objects of a common superclass. It enables the same method to behave differently based on the object calling it.
2. **How does method overriding support polymorphism?**
   Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass, enabling polymorphic behavior.
3. **What is inheritance in Python?**
   Inheritance is a feature that allows a class (child) to inherit properties and methods from another class (parent).

4. Name any two types of inheritance supported in Python.

- Single Inheritance
- Multiple Inheritance

4. **How can you create a class that inherits from another class in Python?**
   By specifying the parent class name in parentheses while defining the child class:

```
class Child(Parent):
    pass
```

5. **What is the purpose of the super() function in Python?**
   The super() function is used to call a method from the parent class, often used to initialize the parent class inside the child class.
6. **How does Python support multiple inheritance?**
   Python allows a class to inherit from more than one class by specifying multiple parent classes in the class definition.

8. What is the difference between single inheritance and multiple inheritance?

- Single Inheritance: One child class inherits from one parent class.
- Multiple Inheritance: One child class inherits from multiple parent classes.

9. What is an abstract class in Python?

An abstract class is a class that cannot be instantiated and usually contains one or more abstract methods.

10. How can you define an abstract class in Python?

By importing the ABC module and using the ABC class:

```
from abc import ABC, abstractmethod

class MyClass(ABC):
    @abstractmethod
    def mymethod(self):
        pass
```

11. What is the purpose of the @abstractmethod decorator?

It is used to declare a method as abstract, meaning it must be implemented in any subclass.

12. What happens if you try to instantiate an abstract class in Python?

Python raises a TypeError since abstract classes cannot be instantiated.

13. What is the role of the init() method in inheritance?

It initializes object attributes and can be overridden in the child class to customize initialization.

14. What is the purpose of the del() method in Python?

It is a destructor method called when an object is about to be destroyed, used to clean up resources.

15. How is constructor invocation different in inheritance?

The child class can explicitly call the parent class constructor using super().__init__().

16. What is composition in Python?

Composition means that a class contains an object of another class and uses its functionalities.

17. What is aggregation in Python?

Aggregation is a special form of composition where the contained object can exist independently of the container class.

18. What is the difference between composition and aggregation?

In composition, the part cannot exist without the whole. In aggregation, the part can exist independently.

19. How can you call a parent class method from a child class in Python?
Using super().method_name() or ParentClass.method_name(self).

```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    def show(self):
        Parent.show(self)
        print("Child method")

c = Child()
c.show()
```

20. What is the purpose of the isinstance() function in Python?

It checks if an object is an instance of a specific class or a subclass thereof.

```
x = 10
print(isinstance(x, int))  # Output: True
class Animal:
    pass
class Dog(Animal):
    pass
d = Dog()
print(isinstance(d, Dog))     # True
print(isinstance(d, Animal))  # True
```

21. How can you convert a basic data type into a user-defined data type in Python?

By passing the basic type as an argument to the user-defined class constructor.

```
class MyNumber:
    def __init__(self, value):
        self.value = value

num = MyNumber(30)  # Converting int to user-defined type
print(num.value)    # Output: 30
```

22. How can you convert a user-defined data type back to a basic data type in Python?

By defining methods like __int__(), __str__(), etc., in the class and using the corresponding type function.

```
class Person:
    def __init__(self, name):
        self.name = name
    def __str__(self):
        return self.name
p = Person("Alice")
print(str(p))  # Output: Alice
```

23. What is a metaclass in Python?

A metaclass is a class of a class that defines how a class behaves.

24. How are metaclasses different from regular classes?

Metaclasses control the creation of classes, while regular classes control the creation of instances.

25. What is the role of unit testing in Python?

It ensures that individual units of code work as expected.

26. What is the purpose of the unittest module in Python?

It provides a framework for writing and running tests to validate code.

27. How can you write a simple unit test in Python?

```
import unittest
class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(1 + 1, 2)
if __name__ == '__main__':
    unittest.main()
```

28. What is the significance of the assert statement in unit testing?

It checks for expected outcomes and raises an error if the condition is false.

29. What is the purpose of the try-except block in Python?

To handle exceptions and prevent program crashes.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

Output:

```
Cannot divide by zero!
```

30. How can you raise an exception in Python?

Using the raise keyword:

```
raise ExceptionType("message")
```

e.g.

```
raise ValueError("An error occurred")
```

31. What is the difference between raise and assert in Python?

- raise is used to trigger exceptions intentionally.

- assert is used for debugging, ensuring conditions are met.

32. How can you create a custom exception in Python?

By subclassing the built-in Exception class:

```
class MyError(Exception):
    pass
```

33. What is hierarchical inheritance in Python?

When multiple child classes inherit from a single parent class.

(e.g., Cat and Dog inherit from Animal).

34. What is the purpose of the str() method in Python?

It defines the string representation of an object.

35. Why can't an abstract class be instantiated directly in Python?

Because it contains incomplete methods that must be implemented in a subclass.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
s = Shape()  # Error: Can't instantiate abstract class
```

36. How does Python handle constructor invocation in multiple inheritance?

Python uses the Method Resolution Order (MRO) to determine which constructor to call.

37. What is the primary purpose of the new() method in Python?

It is responsible for creating a new instance of a class.

```
class MyClass:
    def __new__(cls):
        print("Creating instance")
```

```
        return super().__new__(cls)
    def __init__(self):
        print("Initializing instance")
obj = MyClass()
```

Output:

```
Creating instance
Initializing instance
```

38. What is type conversion in Python?

The process of converting one data type into another, like int(), str() or float().

Explicit Conversion:

```
x = "10"
y = int(x)   # Convert string to integer
print(y + 5) # Output: 15
```

Implicit Conversion:

```
a = 5
b = 2.0
result = a + b  # a (int) is implicitly converted to float
print(result)   # Output: 7.0
```

39. What happens if no constructor is defined in a Python class?

Python provides a default constructor.

40. When is the destructor method (del()) automatically called in Python?

When an object's reference count reaches zero or during garbage collection (execution timing is not guaranteed).